

Code Review Report

This is an engineering review of the whole codebase: architecture, every public Convex function, table rules, auth, locks, and rate limiting.

Contents

- 1. Architecture overview
- 2. Function inventory
- 3. Encapsulation and inheritance
- 4. Table rules
- 5. How Convex works
- 6. Validators and locks

1. Architecture overview

Stack

The app is React 19 with TanStack Start on the frontend and Convex on the backend. TypeScript throughout (tsgo, version 7). Bun is the runner. Vite with the Nitro/Vercel adapter handles SSR.

Browser	Edge / SSR	Data plane
React 19 + TanStack Start (TypeScript tsgo, Tailwind v4 tokens via app.css)	TanStack Router file routes src/routes/*.tsx (+ layout) loaders call Convex queries mutations via useMutation	Convex backend convex/*.ts documents + indexes queries / mutations /
actions	Convex Auth Password provider	authTables (sessions,
accounts)		

File routes. Every file under `src/routes` is a route. `src/routes/service/job.$id.tsx` is the job detail page. `src/routes/admin/audit.tsx` is the audit log. `src/routes/auth/login.tsx` is login. The router tree is generated in `src/routeTree.gen.ts` and never hand-edited. Layouts in `__root.tsx` and `_app.tsx` provide the app shell, auth gating via `useConvexAuth` and `useCurrentUser`, and the inactivity warning modal. Route loaders read `convex/users.ts:me` and `sessionFlags` to force redirects for `mustChangePassword` and `totpExpired`.

Styling. Tailwind v4 tokens live in `src/app.css` as semantic CSS variables: `ink`, `body`, `mute`, `accent`, `bg`, `surface`, `line`, with `.dark` overrides. No hard-coded hex in components.

Backend. One file per domain in `convex/`, plus shared helpers in `convex/lib/`. The full map is below.

Request lifecycle

Here is what happens when someone clicks a button that writes data, say “Generate Final Invoice”:

1. **UI call.** The component calls `useGenerateInvoiceMutation()` from `src/lib/queries.ts`, which wraps `useMutation(api.invoices.generate)`. Reads work the same way: `useQuery(jobQueries.detail(jobId))` subscribes to `convex/jobs.ts:getDetail`.
2. **Convex client.** The Convex React client (set up in `src/lib/convex.ts`) sends the call over WebSocket or HTTP to the deployment, with the auth JWT from Convex Auth attached.
3. **Auth.** The first line of the handler is almost always `await requireActiveSession(ctx, [...])` or `await requireRole(ctx, [...])` or at least `await requireUser(ctx)`. These call `getAuthUserId(ctx)`, fetch the user document, and check `active !== false`, the `lastActiveTs` inactivity window, and 2FA status. On failure they throw a `ConvexError` and the client gets a typed error.
4. **Rate limit.** Right after auth: `await enforce(ctx, class)`. This reads `rateLimits.by_key_window` for the user, class, and current window. If the bucket is full it throws `ConvexError({code: 'RATE_LIMITED', retryAfterMs})`. No writes have happened yet at this point.
5. **Validation.** Zod `Schema.parse(args)` from `src/lib/schemas/*` runs inside the handler. Convex `v.*` validators on the function args run even earlier, before the handler body starts. If either fails, the mutation throws before touching the database.
6. **Business guards.** Things like `findApprovedFinalForJob`, `assertNotLocked`, `canTransition`, plate regex, stock checks, name similarity dedup. Thrown `ConvexError`s become toasts on the client.
7. **Writes.** `ctx.db.insert`, `patch`, or `delete`. Each mutation is one ACID transaction. If any throw happens after a write, the whole thing rolls back. One documented caveat: the best-effort `rateLimitEvents` insert also rolls back, so the durable signal for a rate-limited call is the thrown error, not the event row.
8. **Audit.** `await audit(ctx, action, entity, entityId)` inserts into `auditLogs`, still inside the same transaction, so it commits with the business write.
9. **Return and reactivity.** The return value goes back to the caller. `queryClient.invalidateQueries()` refreshes TanStack Query caches. Convex subscriptions also push updated query results to every subscribed client automatically.

Button click → `useMutation(api.x.y)` → Convex client (+JWT)
 → handler: `require*` → `enforce` → `Zod+v` → business guard → `db.*` → `audit` → return
 → TanStack invalidation + Convex subscription push → UI re-renders

convex/ module map

File	Domain	Exports
<code>convex/schema.ts</code>	schema	All tables, indexes, <code>searchIndexes</code>
<code>convex/lib/auth.ts</code>	auth	<code>getCurrentUser</code> , <code>requireUser</code> , <code>requireRole</code> , <code>isValidRole</code>

File	Domain	Exports
convex/lib/session.ts	session/2FA	heartbeat , requireActiveSession , sessionFlags
convex/lib/audit.ts	audit	audit logging function
convex/lib/rateLimit.ts	throttling	enforce , enforceDedup , windowStartFor , isRateLimitEnabled
convex/lib/invoiceHelpers.ts	invoicing	buildLineItemsForJob , nextInvoiceNumber , assertNotLocked , findApprovedFinalForJob
convex/lib/totp.ts	2FA crypto	TOTP generation, verification, and backup codes
convex/auth.ts	Convex Auth	convexAuth({providers: [Password]})
convex/auth.config.ts	auth config	Auth provider config
convex/http.ts	HTTP router	httpRouter , auth HTTP routes
convex/crons.ts	crons	Rate limit cleanup daily at 03:00 UTC
convex/migrations.ts	migrations	Schema cleanup helpers (legacy fields)
convex/users.ts	users/admin	User queries and admin mutations
convex/twoFactor.ts	2FA flows	TOTP setup, verify, disable, backup codes
convex/customers.ts	service/customers	Customer CRUD with phone dedup
convex/vehicles.ts	inventory	Vehicle CRUD with plate normalization
convex/vehicleBrands.ts	brands	Brand management
convex/parts.ts	parts	Parts inventory and stock movements
convex/jobs.ts	jobs	Service jobs lifecycle
convex/appointments.ts	appointments	Appointment scheduling
convex/invoices.ts	invoicing	Invoice generation, approval, locking
convex/payments.ts	payments	Payment recording
convex/leads.ts	sales/leads	Sales lead management
convex/salesOrders.ts	sales/orders	Sales order lifecycle
convex/deliveries.ts	sales/deliveries	Delivery completion
convex/labourTypes.ts	labour	Labour type management
convex/settings.ts	settings	Global settings (VAT rate)
convex/rateLimit.ts	throttling admin	Rate limit inspection and control
convex/activityLogs.ts	activity	Activity logging and audit
convex/auditLogs.ts	audit	Audit log inspection
convex/seed.ts	demo data	Seed data for development

File	Domain	Exports
<code>convex/backfillPlates.ts</code>	migration	Plate normalization backfill

Zod schemas live in `src/lib/schemas/*` and are imported by both client forms and Convex handlers.

2. Function inventory

This section lists every public function in `convex/` with its auth gate, validators, and rate-limit class.

Shared helpers in `convex/lib/`

Helper	Purpose	Called by
<code>getCurrentUser(ctx)</code>	Fetch current user doc	Auth checkers, audit, rate limit
<code>requireUser(ctx)</code>	Require authenticated user	All queries, most mutations
<code>requireRole(ctx, roles)</code>	Require specific role	Admin/privileged mutations
<code>requireActiveSession(ctx, roles)</code>	Require role + active session + 2FA	All write mutations
<code>sessionFlags(ctx)</code>	Get session state flags	<code>users.me</code> for client gating
<code>heartbeat(ctx)</code>	Update last active timestamp	Called every 60s by client
<code>audit(ctx, action, entity, entityId)</code>	Log to audit trail	Every successful mutation
<code>enforce(ctx, class)</code>	Rate limit check	Every public mutation
<code>enforceDedup(ctx, fingerprint, windowMs)</code>	Deduplication check	<code>payments.record</code>
<code>buildLineItemsForJob(ctx, jobId)</code>	Build invoice line items	Invoice and job mutations
<code>nextInvoiceNumber(ctx, kind)</code>	Get next invoice number	Invoice generation
<code>assertNotLocked(invoice)</code>	Verify invoice not locked	Mutations that modify invoices
<code>findApprovedFinalForJob(ctx, jobId)</code>	Find approved final invoice	Job and payment mutations

`convex/users.ts`

Symbol	Kind	Gate	Rate-limit
<code>me</code>	query	User required	
<code>adminExists</code>	query	User required	
<code>list</code>	query	Admin or audit	
<code>heartbeat</code>	mutation	User required	

Symbol	Kind	Gate	Rate-limit
<code>setRole</code>	mutation	Admin	admin (5/min)
<code>setActive</code>	mutation	Admin	admin (5/min)
<code>bootstrapFirstAdmin</code>	mutation	No prior admin	admin (5/min)
<code>adminResetPassword</code>	mutation	Admin	admin (5/min)
<code>changePassword</code>	mutation	User required	admin (5/min)
<code>clearMustChangePassword</code>	mutation	User required	standard (60/min)

convex/twoFactor.ts

Symbol	Kind	Gate	Rate-limit
<code>status</code>	query	User required	
<code>setup</code>	mutation	User required	admin (5/min)
<code>verifySetup</code>	mutation	User required	standard (60/min)
<code>verifyLogin</code>	mutation	User required	standard (60/min)
<code>disable</code>	mutation	User required	admin (5/min)
<code>regenerateBackupCodes</code>	mutation	User required	admin (5/min)
<code>adminReset</code>	mutation	Admin	admin (5/min)

convex/customers.ts

Symbol	Kind	Gate	Rate-limit	Guard
<code>search</code>	query	User		Text or phone search
<code>get</code>	query	User		Single customer
<code>getWithVehicles</code>	query	User		Hydrate vehicles
<code>create</code>	mutation	Active session	standard	Phone dedup + Levenshtein
<code>update</code>	mutation	Active session	standard	Phone guard if changed

convex/vehicles.ts

Symbol	Kind	Gate	Rate-limit	Guard
<code>get</code>	query	User		
<code>byCustomer</code>	query	User		Index by owner
<code>byPlate</code>	query	User		Normalize + lookup
<code>inventory</code>	query	User		All vehicles

Symbol	Kind	Gate	Rate-limit	Guard
<code>create</code>	mutation	Active session	standard	Plate regex validation
<code>adjustStock</code>	mutation	Active session	standard	Non-negative guard
<code>update</code>	mutation	Active session	standard	Plate if present

`convex/parts.ts`

Symbol	Kind	Gate	Rate-limit	Guard
<code>list/get/search</code>	query	User		
<code>lowStock</code>	query	User		Reorder level filter
<code>categories/brands</code>	query	User		Distinct values
<code>createPart</code>	mutation	Active session	standard	Schema validate
<code>updatePart</code>	mutation	Active session	standard	Schema validate
<code>adjustStock</code>	mutation	Active session	standard	Non-negative, log movement
<code>importParts</code>	mutation	Active session	bulk (5/min)	Loop-validate each row

`convex/jobs.ts`

Symbol	Kind	Gate	Rate-limit	Guard
<code>getDetail</code>	query	User		Hydrate full job
<code>byStatus</code>	query	User		Status filter + order
<code>openCount</code>	query	User		Not completed/paid
<code>dashboardSummary</code>	query	User		7-day trends
<code>byCustomer</code>	query	User		Customer filter
<code>checkIn</code>	mutation	Active session	standard	Vehicle + customer exist
<code>diagnose</code>	mutation	Active session	standard	Status transition
<code>markReady</code>	mutation	Active session	standard	Status transition
<code>complete</code>	mutation	Active session (manager+)	standard	Manager role required
<code>markPaid</code>	mutation	Active session (finance+)	financial (20/min)	Invoice approved + balance
<code>reverseReady</code>	mutation	Active session	standard	No approved final invoice
<code>addJobItem</code>	mutation	Active session	standard	Invoice not locked
<code>removeJobItem</code>	mutation	Active session	standard	Invoice not locked

convex/invoices.ts

Symbol	Kind	Gate	Rate-limit	Guard
<code>getByJob</code>	query	User		Pick approved final
<code>listByJob</code>	query	User		All for job
<code>listBySalesOrder</code>	query	User		All for order
<code>getById</code>	query	User		Single invoice
<code>generate</code>	mutation	Active session (finance+)	financial (20/min)	No approved final
<code>generateSales</code>	mutation	Active session (sales+)	financial (20/min)	Sales domain
<code>regenerate</code>	mutation	Active session (finance+)	financial (20/min)	Not locked
<code>approve</code>	mutation	Active session (finance+)	financial (20/min)	Sets locked=true
<code>createEstimate</code>	mutation	Active session (csr+)	financial (20/min)	Draft estimate
<code>updateEstimate</code>	mutation	Active session (csr+)	financial (20/min)	Draft only
<code>approveEstimate</code>	mutation	Active session (finance+)	financial (20/min)	Estimates stay unlocked
<code>rejectEstimate</code>	mutation	Active session (finance+)	financial (20/min)	Draft to rejected
<code>convertEstimateToFinal</code>	mutation	Active session (finance+)	financial (20/min)	Approved estimate only
<code>adminUnlock</code>	mutation	Admin	financial (20/min)	Sets locked=false

convex/payments.ts

Symbol	Kind	Gate	Rate-limit	Guard
<code>byInvoice</code>	query	User		
<code>record</code>	mutation	Active session (finance+)	financial + 60s dedup	Final + approved + balance cap

convex/appointments.ts

Symbol	Kind	Gate	Rate-limit
<code>list/listRange/upcoming/get</code>	query	User	
<code>create</code>	mutation	Active session	standard
<code>markCheckedIn</code>	mutation	Active session	standard
<code>cancel</code>	mutation	Active session	standard

convex/leads.ts

Symbol	Kind	Gate	Rate-limit
list/search/get	query	User	
create	mutation	Active session	standard
updateStage	mutation	Active session	standard
logFollowUp	mutation	Active session	standard

convex/salesOrders.ts

Symbol	Kind	Gate	Rate-limit
get/list/byVehicle/byLead	query	User	
create	mutation	Active session	financial
complete	mutation	Active session	financial
cancel	mutation	Active session	financial
addPayment	mutation	Active session	financial

convex/deliveries.ts

Symbol	Kind	Gate	Rate-limit
get/getBySalesOrder	query	User	
complete	mutation	Active session (sales+)	financial

convex/labourTypes.ts

Symbol	Kind	Gate	Rate-limit
list	query	User	
create/update/remove	mutation	Active session (manager+)	financial

convex/settings.ts

Symbol	Kind	Gate	Rate-limit
get	query	User	
setVatRate	mutation	Active session (manager+)	financial

Other public surfaces

`convex/rateLimit.ts`: `listEvents`, `getStatus` (admin/audit), `setEnabled` (admin), internal `logEvent` / `cleanup`.

`convex/activityLogs.ts`: `log` (any user), `list` (admin+).

`convex/auditLogs.ts`: `list`, `distinctActions` (admin/audit).

`convex/seed.ts`: `seedData` (mutation), `seed` (action), `checkAuthAccounts` (query).

`convex/vehicleBrands.ts`: `list` (user), `create/update/remove` (manager+, admin rate-limit).

`convex/backfillPlates.ts`: `backfillPlates` (admin only).

`convex/migrations.ts`: `cleanupLegacyJobs`, `cleanupLegacyUsers` (CLI only), `previewLegacyJobs`, `previewLegacyUsers` (read-only preview).

3. Encapsulation and inheritance

TypeScript and Convex do not have classical OOP class inheritance. Encapsulation here is through module boundaries, closure-enforced invariants, and composition.

What encapsulation means

Mechanism	How it works	Example
Module boundary	Only <code>export const</code> is callable from client	Helper functions are private to module
Default-deny auth	Every mutation starts with explicit role guard	<code>requireRole(['admin'])</code> returns false for unauthorized
Composition over inheritance	Guards stack without code duplication	<code>requireActiveSession</code> wraps <code>requireRole</code> + inactivity + 2FA
Lib helpers hide invariants	Locking, window math, TOTP crypto in <code>lib/</code>	<code>assertNotLocked</code> , <code>nextInvoiceNumber</code> in one place
Zod + v validators	Wire-level (<code>v</code>) and domain-level (Zod) contracts	<code>v.union</code> on args + <code>Schema.parse</code> in handler
Component composition in React	Hooks and small components, no class inheritance	<code>useInactivity</code> , <code>useCurrentUser</code> , print layouts

Why not classical inheritance

A textbook OOP design would introduce `abstract class Mutation` with subclasses. In Convex this breaks for three reasons:

1. **Framework constraint.** Handlers must be plain `mutation({args, handler})` values. Classes would hide validators from codegen.

2. **Transaction scope.** Mutations are closures over `ctx`. Subclass dispatch would obscure the atomic boundary.

3. **Composition is clearer.** `await requireActiveSession(ctx, roles); await enforce(ctx, "financial");` is explicit and auditable. Subclass dispatch fixes ordering in one place and breaks silently if a subclass forgets `super.validate()`.

Concrete composition examples:

Auth stacks: `requireActiveSession` → `requireRole` → `requireUser` → `isAuthorized`.

Rate limiting is one-line reuse: `await enforce(ctx, "financial")` calls `getCurrentUser`, `windowStartFor`, checks the shard doc. No base class needed.

Invoice building stacks: `buildLineItemsForJob` and `buildLineItemsForSalesOrder` are standalone helpers reused across `invoices.*` and `jobs.syncInvoiceForJob`. No `InvoiceBuilder` class hierarchy.

What is NOT here: no `extends`, no `super`, no mixins. Encapsulation is narrow public API, private helpers, and composed guards.

4. Table rules

All tables are defined in `convex/schema.ts:defineSchema`. The `authTables` spread comes first.

Core tables

Table	Key fields	Indexes	Write rule
<code>users</code>	email, role, active, totpEnabled, backupCodes	email, by_email	Role check + audit
<code>customers</code>	name, phone, email	by_phone, name search	Dedup on phone/Levenshtein
<code>vehicles</code>	ownerId, plate, make, model, stockQty	by_plate, owner, status	Plate normalize + regex
<code>parts</code>	code, costPrice, sellingPrice, stockQty	by_code, by_brand, by_category	Part-level validation
<code>jobs</code>	vehicleId, customerId, status, complaint	status, by_customer	Status machine via <code>canTransition</code>
<code>invoices</code>	jobId, invoiceNumber, status, locked	by_invoice_number, jobId, salesOrderId	Locked guard after approve
<code>payments</code>	invoiceId, amount, method	by_invoice	Final + approved only
<code>stockMovements</code>	partId, qty, type	by_part	Append-only on stock change
<code>auditLogs</code>	userId, action, entity, entityId	by_entity, by_user, by_ts	Append-only, automatic

Security tables

Table	Key fields	Indexes	Use
rateLimits	key, windowStart, count	by_key_window	Per-user per-class per-minute
rateLimitEvents	userId, actionClass, ts	by_ts, by_action_class	Observability, cleanup cron
activityLogs	userId, event, ts	by_user, by_ts, by_event	User activity tracking
authTables	sessions, accounts, verification codes		Convex Auth managed

Sales/service tables

Table	Key fields	Indexes	Lifecycle
leads	name, phone, stage, interestedVehicleId	stage, name search, phone search	Mutable
salesOrders	vehicleId, leadId, status	by_vehicle, by_lead	Auto-reserve on create
deliveries	salesOrderId, checklist, repld	by_sales_order	One per order
appointments	customerId, status, appointmentTs	by_customer, status, appointmentTs	Scheduled → checked in/ cancelled
labourTypes	name, fixedPrice		Reference table
vehicleBrands	name, normalizedName	by_normalized_name	Uniqueness via normalized

Relation integrity

Convex has no declarative foreign keys. FK discipline is enforced in code:

- `jobs.checkIn` verifies `vehicleId` and `customerId` exist
- `salesOrders.create` checks `vehicleId` and `leadId`
- `appointments.create` checks `customerId`
- `jobs.addJobItem` checks `partId` and `labourTypeId`
- `payments.record` checks `invoiceId`

No delete mutations are exposed on any table, so dangling references are not created.

5. How Convex works

What Convex is

Convex is a reactive backend-as-a-service. You define documents (JSON-like objects) in tables, write server functions, and the client subscribes to query results that push updates automatically. No hand-written REST layer. The deployment runs your `convex/` functions and stores documents in managed storage.

Documents

A document is a JSON object with system `_id: Id<Table>` and `_creationTime: number`. Fields are typed by `v.*` validators. Tables are created by `defineTable({...})` in `convex/schema.ts`. Adding a field is additive. Old documents stay readable via `v.optional`. The `authTables` spread contributes `users`, `authSessions`, `authAccounts` for Convex Auth.

Transactions

Every `query` or `mutation` handler runs in a single ACID transaction. All `ctx.db` calls are atomic: all commit or none do. If a throw happens after a write, the whole transaction rolls back. Concurrency uses optimistic concurrency control (OCC). Convex does not lock rows on read. If two mutations patch the same document concurrently, one commits and the other retries or fails. This matters for rate limiting: a single global counter would become a hot-row bottleneck, so the shipped design shards per `user:class>windowStart`.

Queries vs mutations vs actions

Kind	Reads	Writes	Transactional	Use
<code>query</code>	yes	no	read-only, cached, reactive	All <code>list/search/get/by*</code>
<code>mutation</code>	yes	yes	single ACID transaction	Every <code>create/update/approve</code>
<code>action</code>	yes (via runners)	via mutations only	not across the action	Demo data seeding
<code>internalMutation</code>	yes	yes	transaction, not from client	Cron tasks
<code>httpAction</code> / <code>httpRouter</code>				Auth HTTP endpoints

Reactive subscriptions

A `useQuery(api.jobs.getDetail, {jobId})` is not polling. The client opens a subscription. When any document read by that query changes, Convex pushes a fresh result to all subscribers. Dashboard and job detail stay consistent without manual refetch.

Scheduler and crons

At 03:00 UTC daily, the cleanup mutation scans rate limit and event tables, deletes old rows. No per-mutation scheduler on hot path.

Auth

`convex/auth.ts` configures `convexAuth({providers: [Password]})`. User identity in any handler is `getAuthUserId(ctx)`. The `users` table holds profiles plus `totp*`, `lastActiveTs`, `mustChangePassword`. Password hashes live in `authAccounts.secret` (Scrypt), never in `users`.

Client mapping

Codegen via `bunx convex codegen` produces `convex/_generated/dataModel.ts` and `api.ts`.

6. Validators and locks

Validator layers

Convex `v` runs before the handler body. Each function declares `args: { field: v.xxx }`. Mismatch returns error to client without entering the transaction.

Examples:

- `jobs.addJobItem` declares `type: v.union(v.literal('part'), v.literal('labour'))`, `partId: v.optional(v.id('parts'))`.
- `payments.record` declares `method: v.union(v.literal('cash'), v.literal('transfer'), ...)`. Enum enforced at wire.
- `parts.adjustStock` derives validators from single `src/lib/enums.ts` source so `v` and Zod stay aligned.

Zod runs inside handler after auth. Most mutations call `Schema.parse(args)` from `src/lib/schemas/*`. This is the domain-level contract: trims, regexes, bounds that `v` alone cannot express. The handler throws `ZodError` if Zod fails, still before `ctx.db.*` write.

Pattern always: `require*` → `enforce` → `Schema.parse` → business guard → `ctx.db.*` → `audit` → return.

Invoice lock mechanism

Data field: `invoices.locked?: boolean`. Absent means false.

Trigger 1 - Locking at approve: `invoices.approve` is the only handler that sets `locked:true`. Not `approveEstimate`. Just `approve`. Before the patch, the handler checks invoice is not already approved and is not an estimate. Estimates use `approveEstimate` which patches `status:'approved'` but does NOT lock.

Trigger 2 - Read as guard: The lock is read at these sites:

- `invoices.generate/regenerate` check `existingFinal` via `assertNotLocked(existingFinal)` or `paid`
- `invoices.updateEstimate` checks `assertNotLocked(invoice)`
- `jobs.addJobItem/removeJobItem` check `findApprovedFinalForJob` non-null
- `payments.record` rejects `locked && paid`, plus checks balance cap
- `jobs.reverseReady` checks `findApprovedFinalForJob` non-null
- `invoices.convertEstimateToFinal` checks no existing approved final

The helper `findApprovedFinalForJob` returns `kind==='final' && approved`. An approved final is treated as locked even if `locked` field was forgotten. The code double-guards via both `assertNotLocked` and `allInvoices.some(i=>i.locked)`.

Unlock path: `invoices.adminUnlock` is the only mutation that patches `locked:false`. It requires `requireRole(['admin'])`, a reason 10-300 chars, and only works if invoice is actually locked. After unlock, the final becomes `approved:true, locked:false` so further writes are allowed until next `approve`.

generatedById tracking: On first `generate` and on `approve`, the final records which user produced or approved it. Queries project `generatedById` only for admin. Non-admin users receive invoice without that field.

Rate-limit classes

Every public mutation throws `RATE_LIMITED` when per-user per-class budget is exhausted. Classes are wired in each mutation (see section 2):

```
await requireActiveSession(ctx, [...])
await enforce(ctx, "financial") // or "admin" | "bulk" | "standard"
const parsed = SomeSchema.parse(args)
```

Classes:

- **admin 5/min:** Users, two-factor setup/disable, vehicle brands, backfill
- **financial 20/min:** Payments, invoices, sales orders, labour types, deliveries, VAT settings
- **bulk 5/min:** Parts import
- **standard 60/min:** Customers, vehicles, leads, appointments, most job mutations, parts CRUD

Queries never call `enforce`. They are read-only and limiting them hurts busy shifts.

Dedup guard: `payments.record` calls `enforceDedup(ctx, 'pay:${invoiceId}:${amount}:${method}')` additionally. An identical payment within 60 seconds throws even if the 20/min bucket permits. Prevents double-click ledger noise.

Window math: `windowStartFor(now, 60000) = floor(now/60000)*60000`. First mutation in a window inserts `{key, windowStart, count:1}`. Later mutations patch `count+1`. At `count >= limit` throw includes `retryAfterMs = ws+60000 - now` for client countdown. Next wall-minute gets fresh doc, no contested reset. Cron at 03:00 UTC prunes `windowStart < now-24h` and old events.

Session expiry

`requireActiveSession` throws when `now - lastActiveTs > 30*60*1000`. On client, `useInactivity` debounces events, heartbeats `api.users.heartbeat` every 60 seconds, shows warning at 25 minutes, hard-redirects to `/auth/login?expired=1` at 30 minutes. `users.me` returns `sessionFlags` so router gates `/auth/verify-2fa` and `/auth/change-password`. Until first heartbeat, server allows writes. Once set, every `requireActiveSession` write respects the 30-minute clock. 2FA adds its own `lastTotpVerifiedTs` gate with the same window.

Document ID: CMA-CRR-004 | **Date:** 01 September 2026